UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Graph Neural Networks

Lecture 16

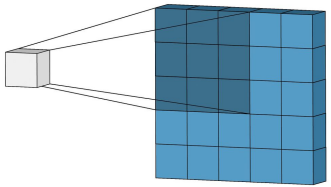
Course of:

Statistics and data analysis

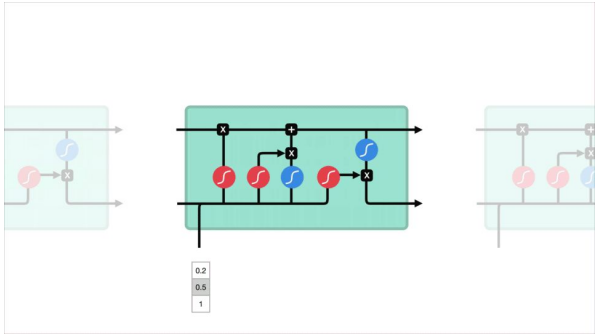
Federico De Guio - Martino Borsato

Data **representations**

- In deep learning, **tailoring algorithms to the structure (and symmetries) of the data** has led to groundbreaking performance
  - **CNNs** for images
    - Fixed mesh arrays of data



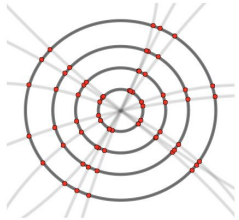
  - **RNNs** for analysis of time series and language processing
    - sequences

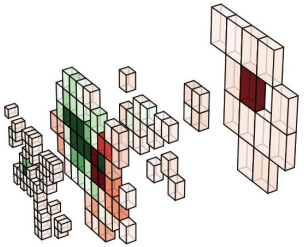


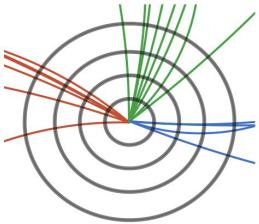
  - Can we do better for **sparse events** like jets?

Properties of physics data

- Measurements distributed in space (and time) **irregularly**
- **Sparse** (most detector channels are empty), but pockets of density
- **Complex interdependencies** between measurements
- Physics “objects” composed of **multiple measurements**
- Inherent symmetries (Lorentz boosts, rotational)







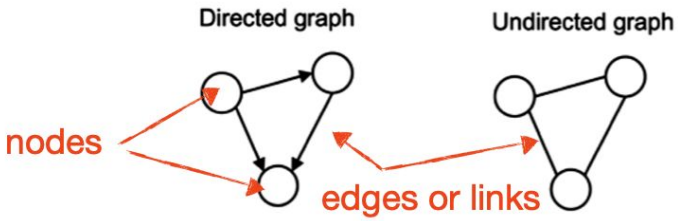
- **Graph embedding** of the data can handle these properties

Formalizing a **graph**

Gentle introduction to GNN

- Graphs are a kind of data structure that is **nonlinear and abstract**, able to adapt to powerful representation of different physical and social systems

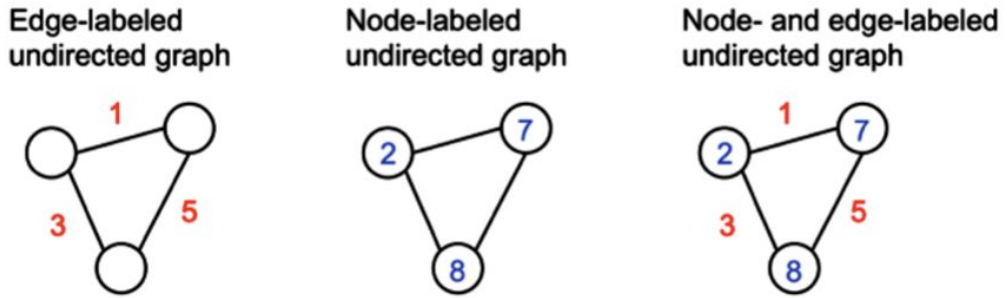
- Common types of graphs:



Graphs are typically sparse: only a small subset of all possible edges are present

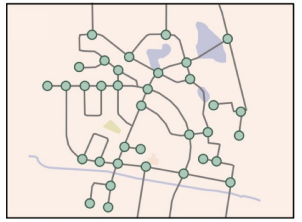
Fully connected graphs are called **sets** (unordered lists)

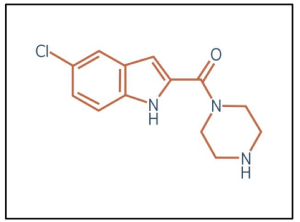
Features are triplets of global features, node features, edge features

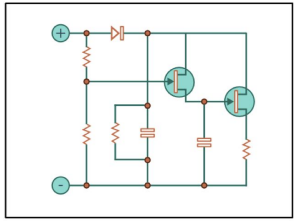


Examples of graphs

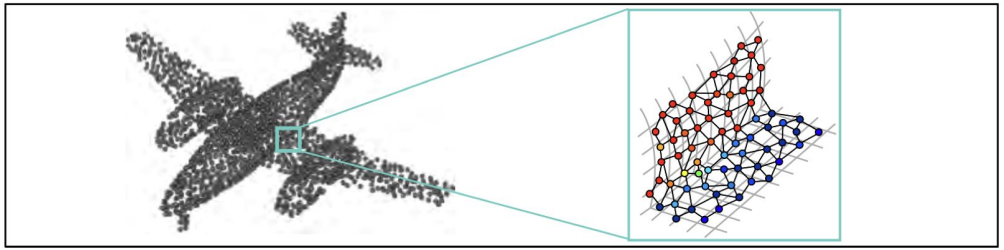
- Many objects in the real world are **naturally represented by graphs**
  - road networks
  - electrical circuits or electrical networks
  - chemical molecules
  - physics objects measured in HEP detectors (jets reconstructed in calorimeters, charged particle tracks reconstructed in tracking detectors, …)
  - interactions between objects (particles, molecules, proteins, humans, planets, gravitational objects, …) can be expressed as graphs, where the nodes are the objects, and there is an edge between two objects if they interact
  - geometrical 2D/3D point-clouds (geometrical deep learning)
  - social networks: are graphs where nodes are people and the edges represent friendships between them
  - wikipedia can be thought of as a graph in which the nodes are articles and the edges represent hyperlinks between articles







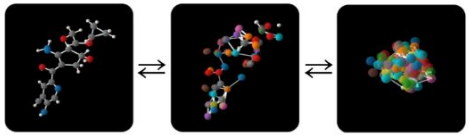
- 3 examples in the following slides

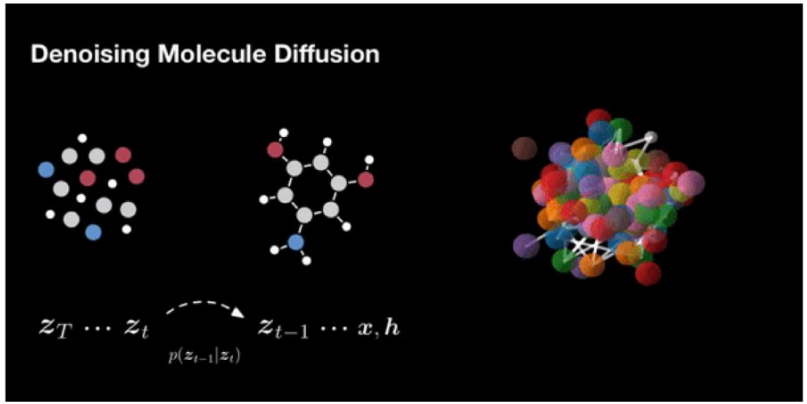


Graph-based diffusion models for drug design

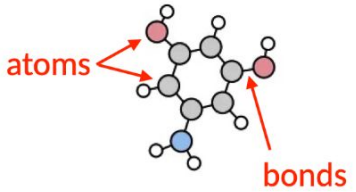
- New ways to the molecular design implemented via an iterative refinement process that allows methodic design of molecules and correction of possible errors arising during the process

noise is gradually injected until the molecule becomes a cluster of random atoms. At each step, an inverse process $p(z_{t-1}, z_t)$ is learned, which tries to predict the noise that was added to the data and then removes it to obtain clean data





graph neural network architecture: to naturally represent the intrinsic topological structures of molecules.

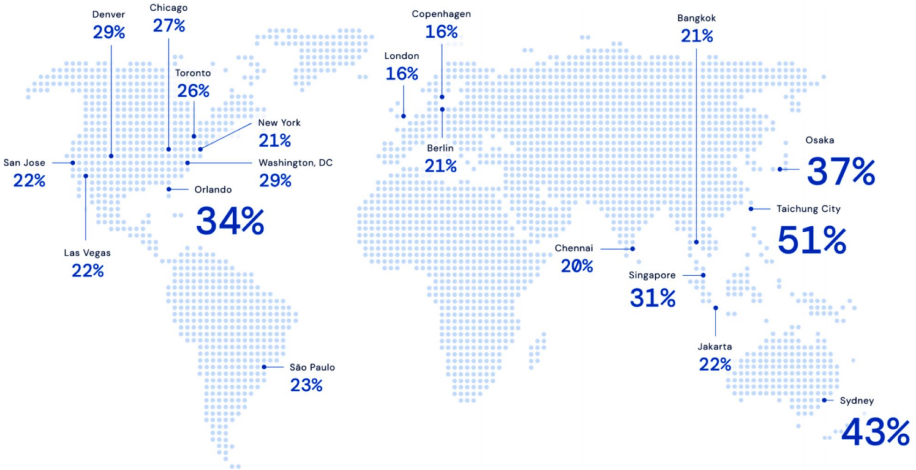


physics priors: incorporated into the neural network model in such a way that it respects the physical symmetries of molecules (rotations, translations, and reflections), to facilitate learning

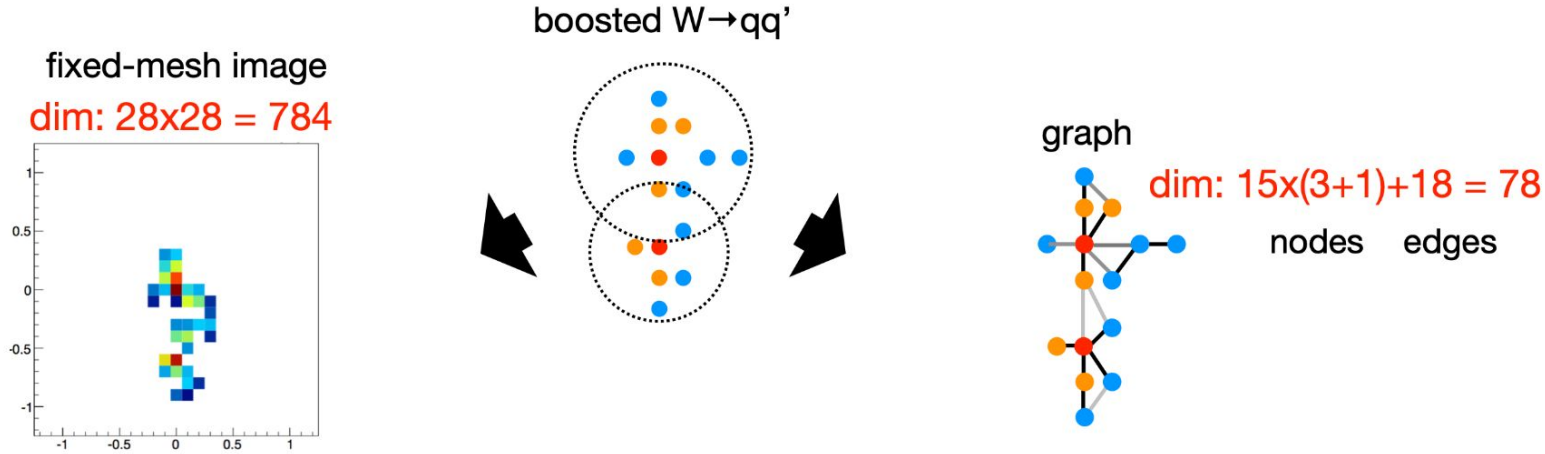
GNN for traffic forecasting

- Goal: **improving ETA predictions in Google Maps**
- Accurately predicting the estimated time of arrival (ETA) for a given route requires a complex understanding of the spatiotemporal interactions taking place on the road
  - GNNs are well suited for this task because **roads and their intersections naturally form a graph** network
  - A GNN-based system reduced negative ETA outcomes between 16% and 51% around the world in live production

- first, roads are chunked into connected segments that follow typical traffic routes and form longer super-segments
- the world is divided into regions that have similar driving behaviors and train region-specific GNNs
- data represents the actual traversal times across segments and super-segments, which are used as node-level and graph-level labels for prediction, respectively
- for a given starting time, the GNN learns the travel time of each supersegment at specific points in the future



Enhanced data representation of **jets with graphs**



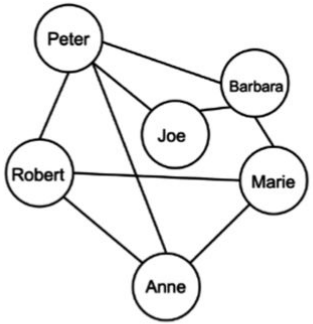
- Image-based: treating the energy deposition on each calorimetric cell as a pixel intensity creating an image of the event in fixed-shape mesh
  - natural representation for Convolutional Neural Networks
  - unclear how to incorporate additional information associated to each energy deposit
  - very sparse and inefficient representation: jets decays have O(10) to O(100) particles → more than 90% of the pixels are empty

- Point cloud-based: unordered sets of entities distributed irregularly in space, analogous to the point cloud representation of 3D shapes
  - clouds allow rich internal structures
  - easy to incorporate additional information associated to the energy deposit (cell type, energy, timing information, …)
  - the architecture of the neural network has to be carefully designed to fully exploit the potential of this representation → Graph Neural Networks

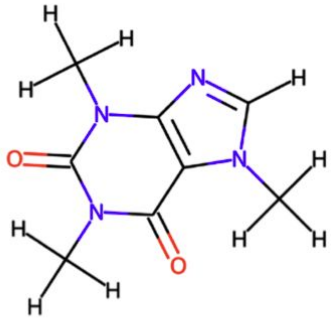
Graph connectivity: **undirected** graphs

- An undirected graph consists of nodes that are connected via edges where the **order of the nodes and their connection does not matter**

- The graph is defined as a pair (V, E), where V is a set of the graph's nodes, and E is the set of edges making up the paired nodes
- The graph structure can be encoded as a |V|×|V| **adjacency matrix A**. Each element x_{ij} in the matrix A is either a 1 or a 0
- **Undirected graph means A is symmetric ($x_{ij} = x_{ji}$)**



Friend graph



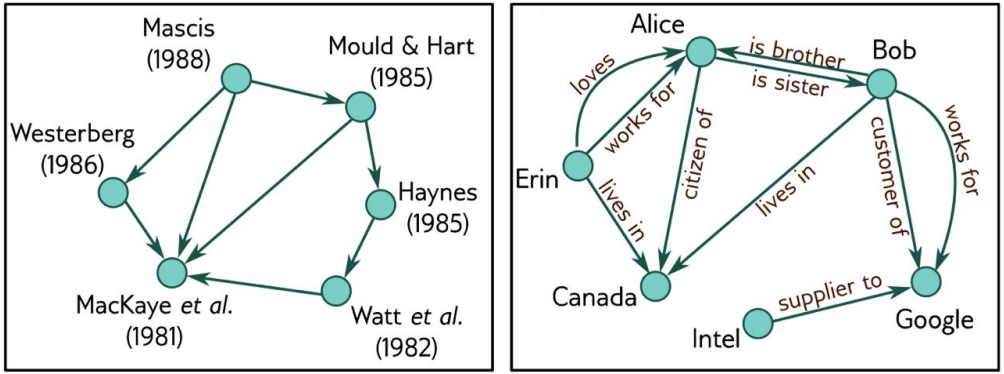
Molecular graph of caffeine

- Common examples of data that can be represented as undirected graphs include images, protein-protein interaction networks, point clouds, …

Graph connectivity: **directed** graphs

- Directed graphs connect nodes via **directional edges**
- Defined in the same way as an undirected graph, except that E, is a set of ordered pairs
  - e.g. $x_{ij} \neq x_{ji}$

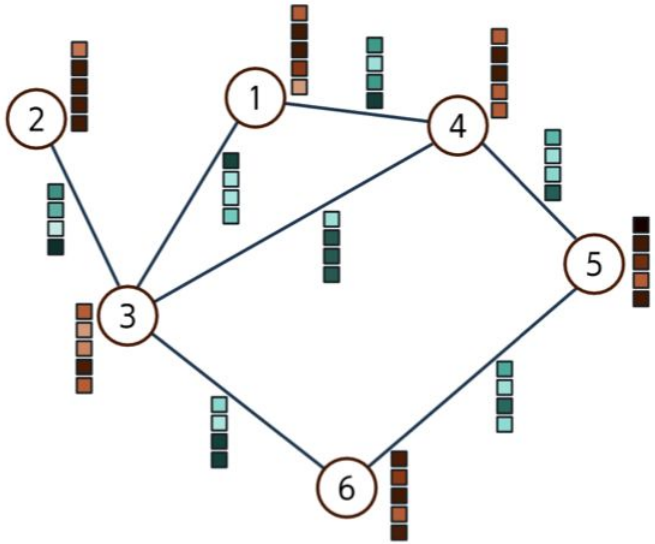
- Examples are:
  - **Citation network**, where nodes are publications and edges from a node are directed toward the nodes of papers that a given paper cite
  - **Knowledge graph** with multiple edges representing different relations between each node

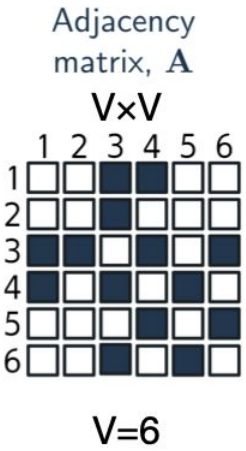


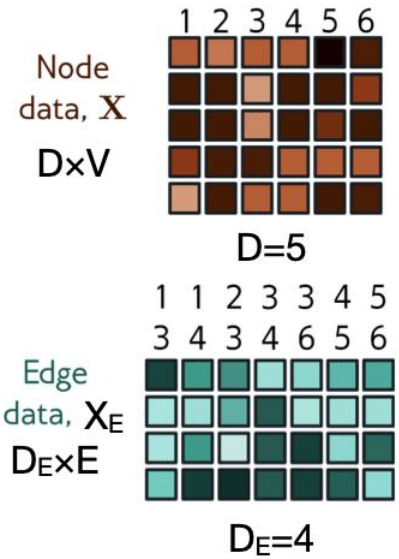
Encoding the information in a graph

Embeddings are the features relative to nodes and edges

- A graph G=(V ,E) is represented by 3 matrices:
  - **adjacency matrix** A (dim: VxV) representing **graph structure**
  - **node label matrix** X (dim: VxD) representing the **node embedding**
  - **edge label matrix** XE (dim: ExD_E) representing the **edge embedding** (often simpler graph don't use X_E)

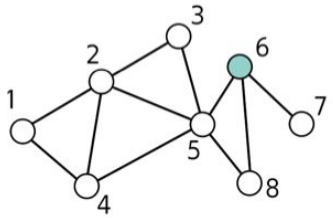






Properties of the **adjacency matrix**

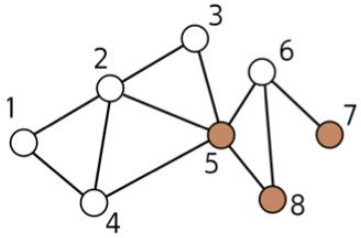
- The adjacency matrix can be used to to **find the neighbors of a node using linear algebra**
  - Example: encode the j-th node as a one-hot V-size column vector with all zeros but the j-th element



$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

6th node

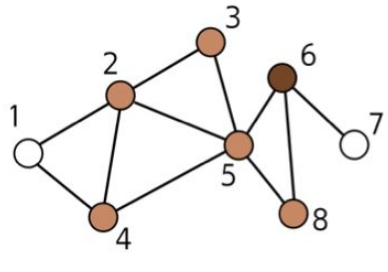
$$\mathbf{Ax} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$



6th node - neighbours

$$\mathbf{A}^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 2 & 0 & 0 & 0 \\ 1 & 4 & 1 & 2 & 2 & 1 & 0 & 1 \\ 1 & 1 & 2 & 2 & 1 & 1 & 0 & 1 \\ 1 & 2 & 2 & 3 & 1 & 1 & 0 & 1 \\ 2 & 2 & 1 & 1 & 5 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 3 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 2 \end{bmatrix}$$

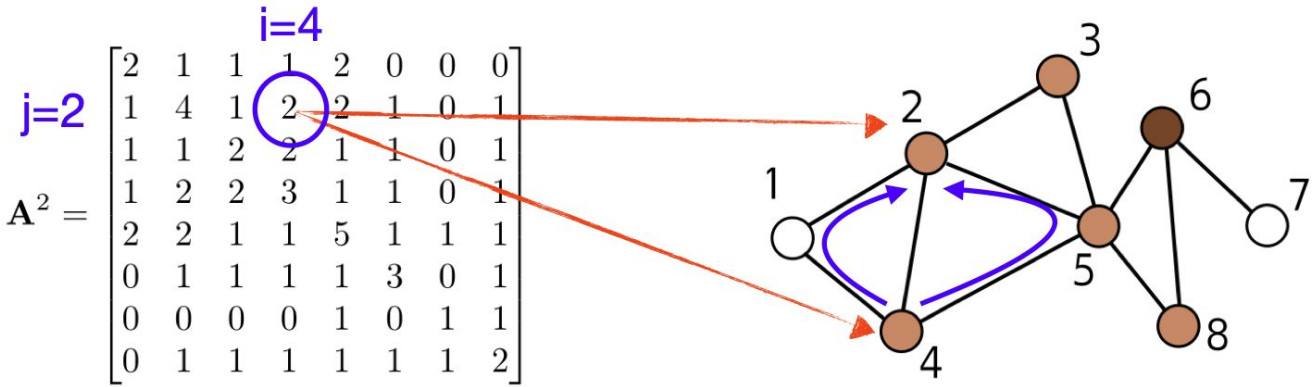
$$\mathbf{A}^2\mathbf{x} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 3 \\ 0 \\ 1 \end{bmatrix}$$



6th node walks of length 2 to the other nodes of the graph

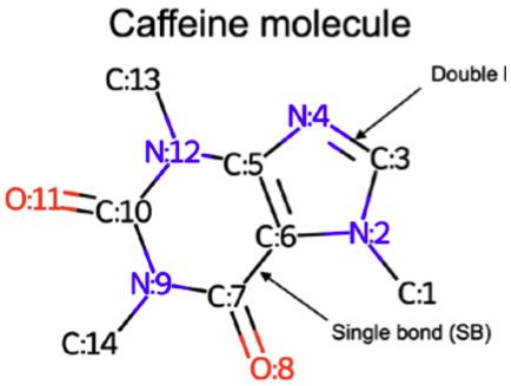
Properties of the **adjacency matrix**

- In general the element (i, j) of $\mathbf{A^m}$ contains the **number of unique "walks" of length m** from node i to j



One example: the caffeine molecule

- We want to encode the **connections between atoms** the **atom types** and the **bond types** between them
  - Adjacency matrix to encode the connection structure between atoms
  - Node label matrix to represent the type of atom (**node embeddings**)
  - Edge label matrix to represent the bond type (**edge embeddings**)



$A =$

Adjacency matrix

$$\begin{bmatrix}
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0
\end{bmatrix}$$

$X =$

Node label matrix

One-hot encoding of the atom type

C	N	O
1	0	0
0	1	0
1	0	0
0	1	0
1	0	0
1	0	0
1	0	0
0	0	1
0	1	0
1	0	0
0	0	1
0	1	0
1	0	0

$X_E =$

Edge label matrix

One-hot encoding of the bond type

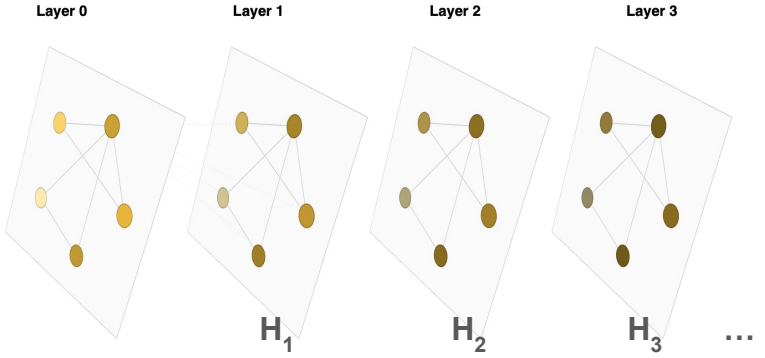
	SB	DB
(1,2)	1	0
(2,3)	1	0
(2,6)	1	0
(3,4)	0	1
(4,5)	1	0
(5,6)	0	1
(5,12)	1	0
(6,7)	1	0
(7,8)	0	1
(7,9)	1	0
(9,10)	1	0
(9,14)	1	0
(10,11)	0	1
(10,12)	1	0
(12,13)	1	0

Graph Neural Networks

- A GNN is a model that takes the node embeddings X and the adjacency matrix A as inputs and **passes the graph through a series of K layers**
- **At each layer the node embeddings are updated** to create intermediate "hidden" representations of the graph H_k, before finally computing output embeddings

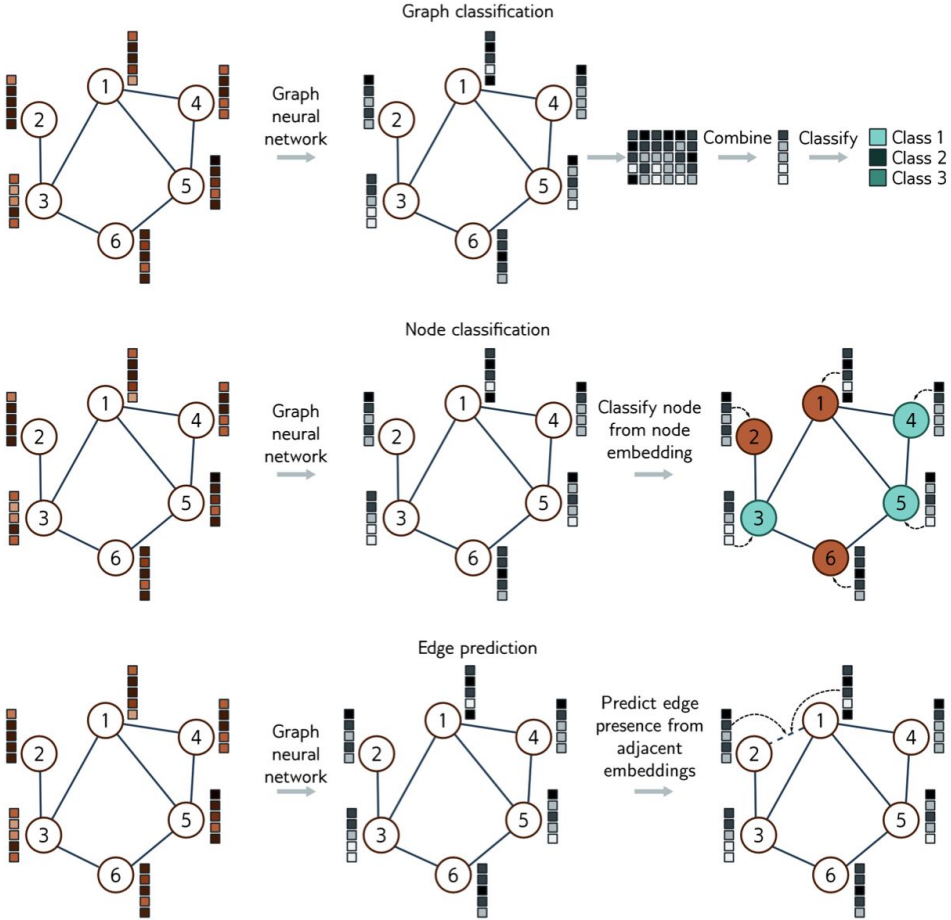
$$(H_1, A) = f_1(X, A)$$
$$(H_2, A) = f_2(H_1, A)$$
$$\vdots$$
$$(H_k, A) = f_k(H_{k-1}, A)$$
$$\vdots$$
$$(H_K, A) = f_K(H_{K-1}, A) = f_K(f_{K-1}(\cdots f_1(X, A))))))$$

$f_k = f_{W_k}$ w/ W_k learnable weights



- At the start, each column of the input node embeddings X contains **only the information about the node itself**
- At the end, each column of the model output H_K contains **information about the node and its context within the graph**

GNN tasks



- **Whole graph classification**:
  - obtain an embedding for the entire graph by performing a global pooling operation and use the embedding on a MLP
  - $y = \mathrm{softmax}[b_K + W_K H_K 1/N]$

Mean pooling

- **Node classification:**
  - is it possible to classify a single or every node, or any combination of it
  - $y(i) = \mathrm{softmax}[b_K + W_K h_K^{(i)}]$

- **Edge classification/link prediction:**
  - a prediction of an edge can be obtained by combining the features of the corresponding nodes, and pass this information to an MLP
  - $y^{(i,j)} = \mathrm{softmax}[h_K^{(i)T} h_K^{(j)}]$

Properties of GNNs

- Must have:
  - Graph-level outputs should be **invariant under permutations of the nodes**
    - $f(PV) = f(V)$ → Node ordering should not matter
  - Node- or edge-level outputs should be **equivariant under permutations of the nodes**
    - $f(PV, PAP^T) = Pf(V, A)$ → Outputs should be permuted if inputs are permuted
  - Learning uses "**locality**"
    - outputs of nodes in the same neighborhood should be more similar

Ensures coherence
→ relations between nodes are learned, not their position in the list

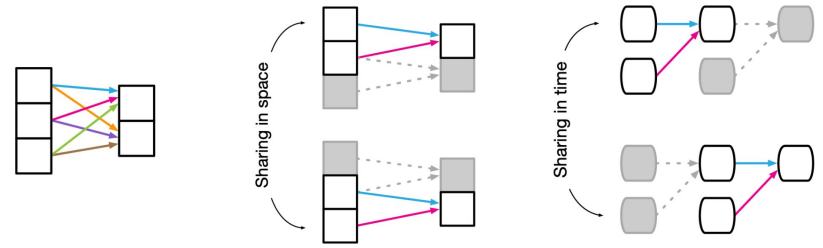
- Nice to have:
  - Account for nodes of different types ("heterogeneous graphs")
  - Generalizes to smaller/larger graphs than those seen in training
  - Computationally efficient

Inductive biases

- An inductive bias expresses **assumptions about the data-generating process** or the **space of solutions**, allowing a learning algorithm to prioritize one solution over another
  - GNNs give priority to relational properties and discard solutions that do not comply with the graph structure

Component	Entities	Relations	Rel. inductive bias	Invariance
Fully connected	Units	All-to-all	Weak	-
Convolutional	Grid elements	Local	Locality	Spatial translation
Recurrent	Timesteps	Sequential	Sequentiality	Time translation
Graph network	Nodes	Edges	Arbitrary	Node, edge permutations

- Inductive biases can improve the search for solutions and find solutions that generalize
  - However, **mismatched inductive biases can also lead to suboptimal performance** by introducing constraints that are too strong
- **Relational inductive bias:** explicit representation of entities and relations

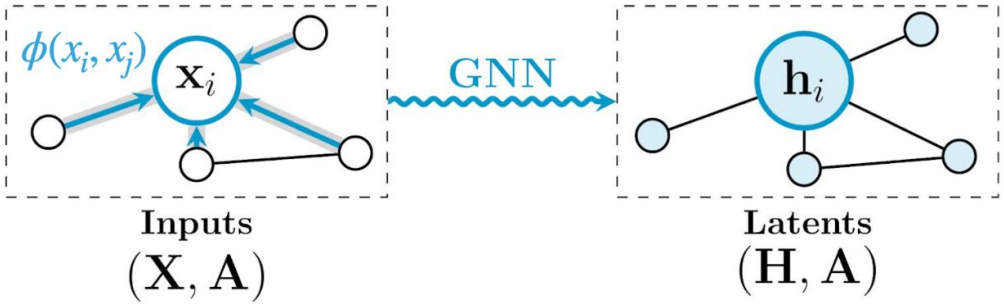


(a) Fully connected (b) Convolutional (c) Recurrent

Learning over a graph: **graph convolution**

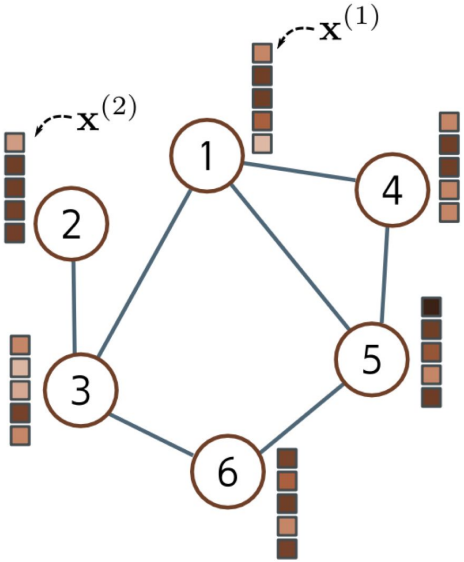
- In the simplest implementation, the convolutional approach seen in CNNs is generalized to a **spatial-based graph convolutional neural networks** (GCN)
  - update each node by aggregating information from nearby nodes → a relational inductive bias is induced and info from neighbours is prioritized
  - are spatial-based because they do this in a straightforward manner using the original graph structure
  - the convolutional approach is even more important for graphs than images because it can function with graphs of different sizes

- **Key ingredient is the message passing step**



Graph convolution

- Let's consider a graph described by a V×V **adjacency matrix A** and V×D **node feature matrix X**
- Graph convolutions **update the embeddings at each node** based on their own embeddings and the ones of their neighbors via a **message passing** operation



1. **Aggregation phase**: at each node i in layer k, information from neighboring nodes is aggregated by summing their node embeddings h:

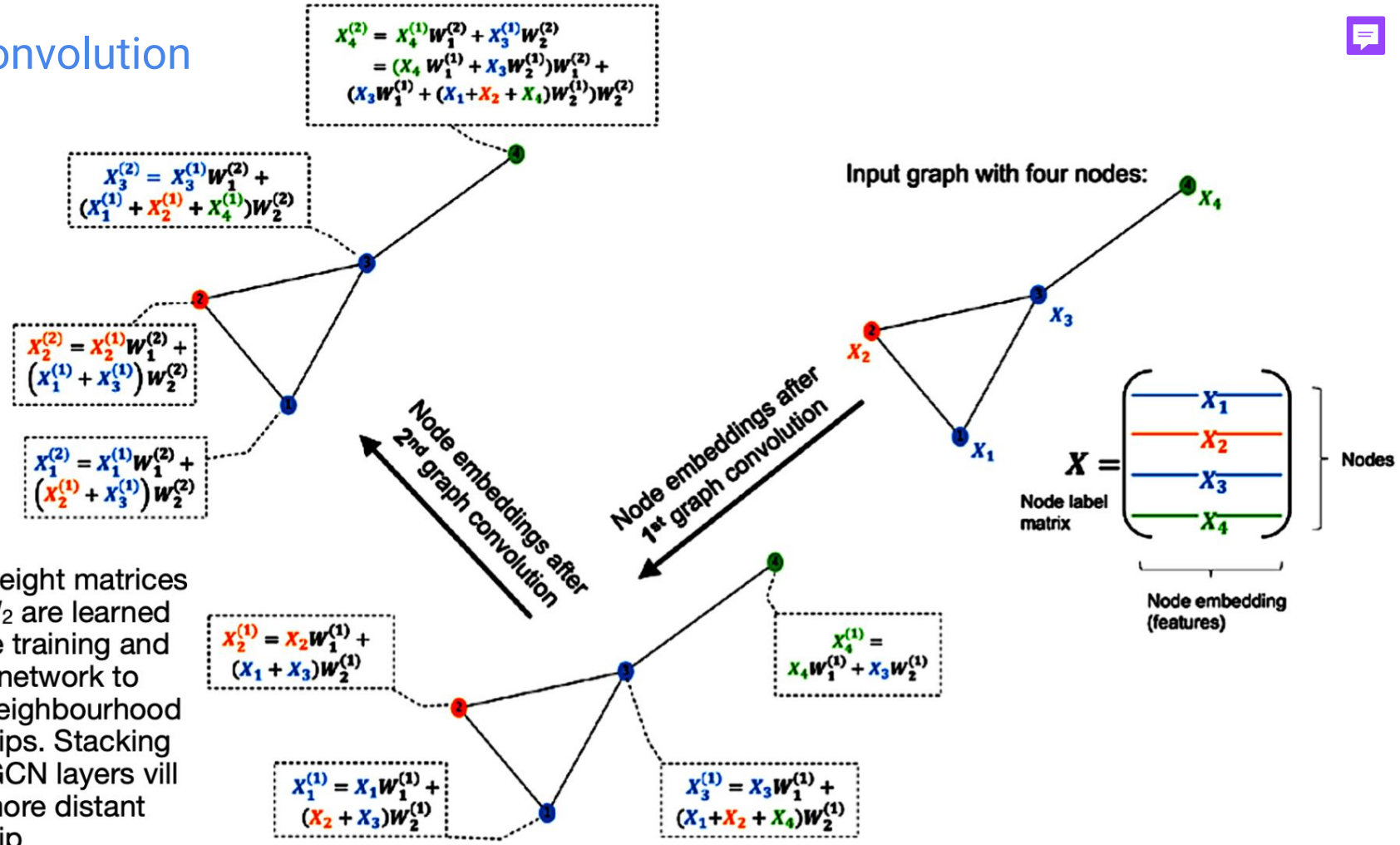
$$AGG_i^{(k)} = \sum_{j \in N(i)} W_2^{(k)} h_j^{(k)}$$

2. **Update phase**: applies a linear transformation to the current node and to the aggregated message and pass the result through a non linear activation:

$$h_i^{(k+1)} = \phi[b^{(k)} + W_1^{(k)} h_i^{(k)} + AGG_i^{(k)}]$$

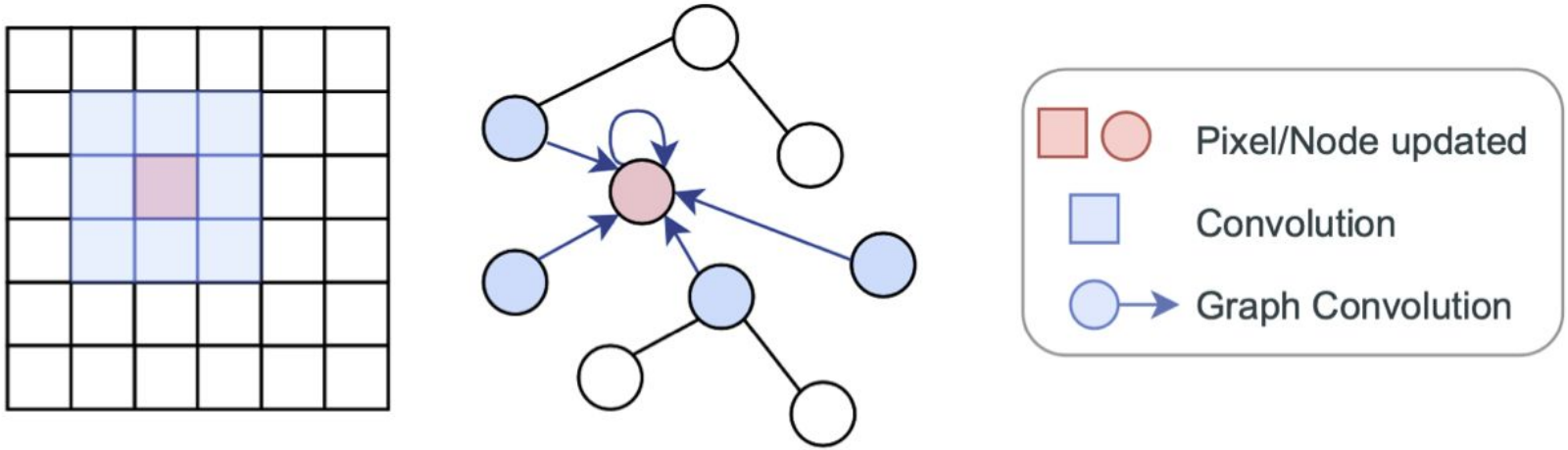
non linearity induced via the activation function ϕ

Graph convolution



- the two weight matrices W_1 and W_2 are learned during the training and allow the network to capture neighbourhood relationships. Stacking multiple GCN layers vill capture more distant relationship

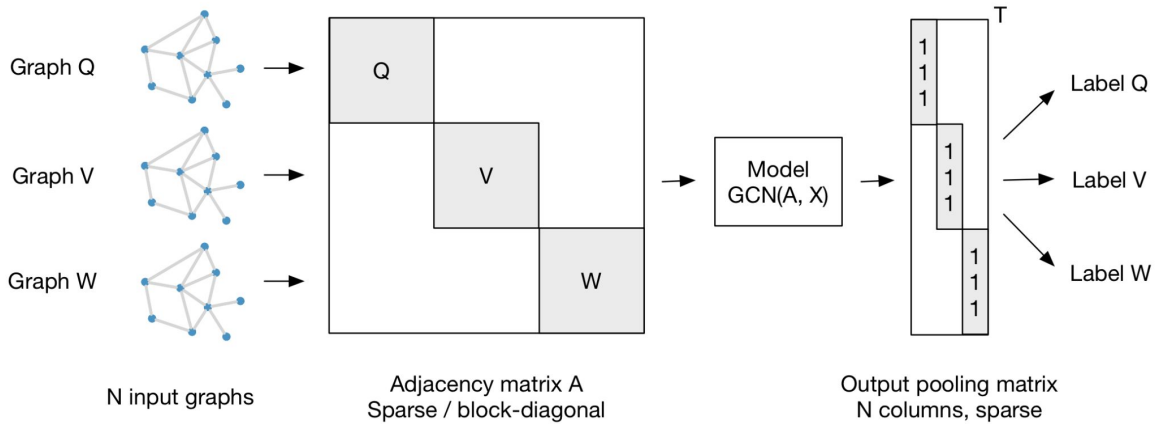
Graphs vs CNNs convolution



Batching multiple graphs

- MLP, CNN leverage the parallelism of modern GPU hardware to concurrently process an entire batch simultaneously
  - However **in GNN each graph may have a different number of nodes** → the matrices X_i and A_i have different sizes → not possible to concatenate them into 3D tensors

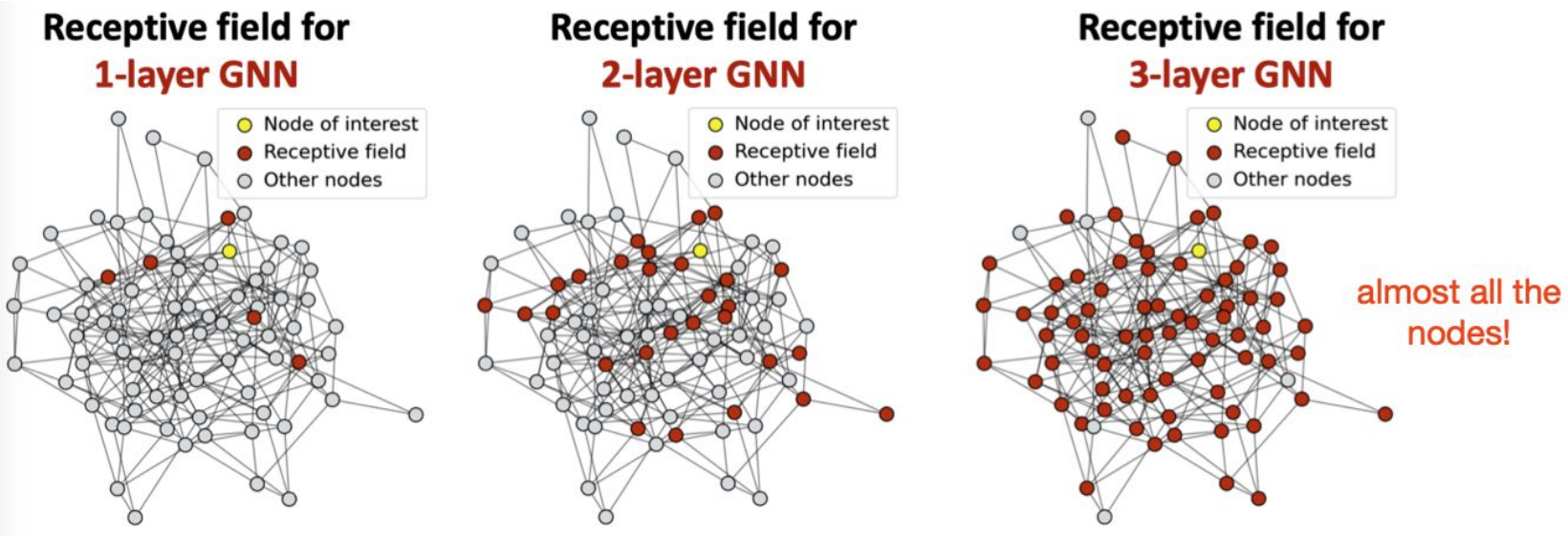
- Solved by treating the **graphs in the same batch as disjoint components of a single large graph**
  - The network can then be run as a single instance of the network equations
  - The mean pooling is carried out only over the individual graphs to make a single representation per graph that can be fed into the loss function



Over-smoothing in GNNs

- A common issue with GNNs is the so called over-smoothing: after several iterations of GNN message passing **all the nodes embedding converge to the same value**
  - common in basic GNN models
  - related to the **receptive field** of the GNN → if 2 nodes have highly overlapping receptive fields their embeddings will be very similar

in a K-layers GNN, each node has a receptive field of K-hop neighborhood



How to avoid over-smoothing problems

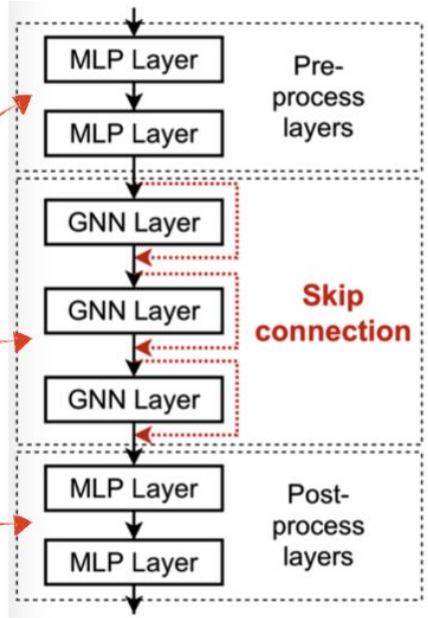
- Analyze the necessary receptive field by **computing the diameter of the graph** and
  - Set number of GNN layers just slightly larger than the needed receptive
  - Refrain to set it to unnecessarily large values

- To increase the expressive power of the GNN with a small number of layers:
  - Add layers that do not pass messages
  - Message passing based on deep MLP networks
  - Use of skip connections (concat features)

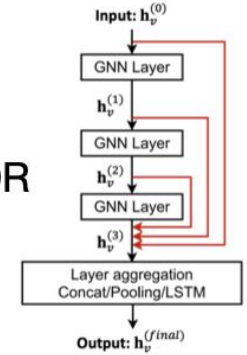
pre-processing layers: used when encoding node features is necessary (as ex. when nodes represent images/text)

skip connections: mix shallow and deep GNNs, so that residual networks works like ensemble of shallow networks

post-processing layers: used when reasoning / transformation over node embeddings are needed
example: graph classification, knowledge graphs



OR



Example…

Classification of particle jets using GNNs - multiclass classification

- **Jet tagging** at the Large Hadron Collider is a critical data analysis technique used to identify the originating particle of a jet
  - A jet is a collimated spray of particles produced in high-energy collisions
- Except for quark- (“q”) and gluon-initiated jets (“g”), all other jets in the dataset have multiple "prongs"
  - The analysis of the structure of the jet provides info on the jet type
- → **Implement a ML approach that uses GNN to perform multiclass classification** and compare performance with CNN based approaches

